Course Name: Advanced Programming (OOP) Lab

Project Title: Multiplayer Turn-Based Game Engine using Java

Student Name: Ritisha Mondal

Roll Number: 79

Instructor Name: Soumadip Biswas

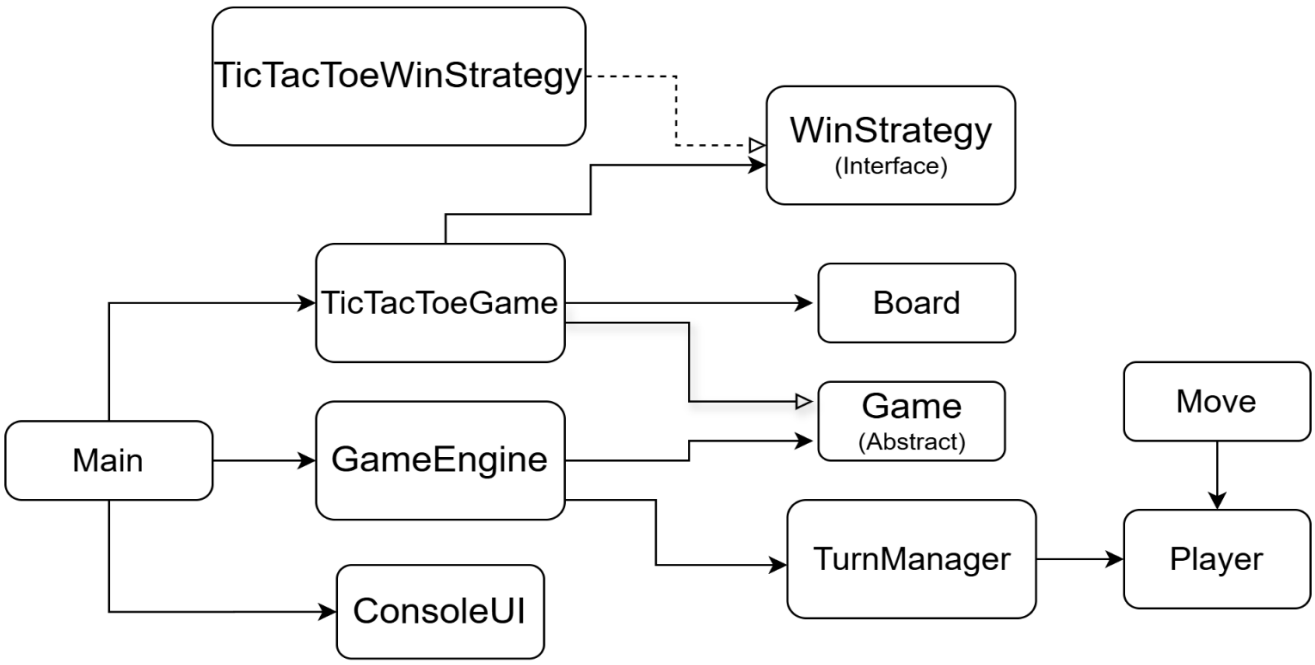
Spring 2026

1. PROBLEM STATEMENT:

Code reuse is challenging in many turn-based games because game logic is closely tied to particular rules. This project creates a reusable Java Turn-Based Game Engine that distinguishes between game-specific logic and game flow (players, turns, state management). The Strategy Pattern and object-oriented concepts like abstraction, inheritance, and polymorphism make it simple to extend the system to support a variety of games. The objective is to develop an architecture that is extensible, maintainable, and modular.

2. SYSTEM DESIGN:

(A) UML CLASS DIAGRAM



(B) DESCRIPTION OF MAJOR CLASSES:

- *Game (Abstract Class): Establishes the framework for every game*
- *TicTacToeGame: Uses logic from Tic-Tac-Toe*
- *GameEngine: Manages the flow of game execution*
- *TurnManager: Manages player turns*
- *Player: Keeps player information*
- *Move: Indicates a movement*
- *Board: Upholds the game grid*
- *WinStrategy: Winner logic interface*
- *ConsoleUI: Manages user communication*

(C) PACKAGE STRUCTURE:

app/

engine/

model/

games/tictactoe/

strategy/

ui/

exception/

(D) INTERACTION FLOW:

User → ConsoleUI → GameEngine → Game → Strategy → Result

3. OOP CONCEPTS DEMONSTRATED:

- **Abstraction**

```
public abstract class Game {
   public abstract void makeMove(Move move);
}
```

Where used: Base class for all games
Why used: To define common structure
Explanation: Ensures all games implement move logic

- **Inheritance**

```
public class TicTacToeGame extends Game
```

Where used: Game specialization
Why used: Code reuse
Explanation: Extends base functionality

- **Polymorphism**

```
game.checkWinner();
```

Where used: Method overriding
Why used: Different logic for different games
Explanation: Same method, different behaviour

- **Exception Handling**

```
if (!board.isCellEmpty(row, col)) {
   throw new InvalidMoveException("Cell occupied");
}
```

Where used: Move validation
Why used: Prevent invalid actions
Explanation: Ensures safe execution

- **Collections**

```
List<Player> players = new ArrayList<>();
```

Where used: Store players
Why used: Dynamic data handling
Explanation: Allows flexible player list

4. IMPLEMENTATION HIGHLIGHTS

Snippet 1: Move Validation

```
if (row < 0 || row >= 3) {
    throw new InvalidMoveException("Invalid position");
}
```

Prevents invalid input

Snippet 2: Strategy Pattern

```
Player winner =
winStrategy.checkWinner(board.getGrid());
```

Separates rules from logic

Snippet 3: Draw Detection

```
if (isBoardFull()) {
   gameState = GameState.DRAW;
}
```

Handles game end condition

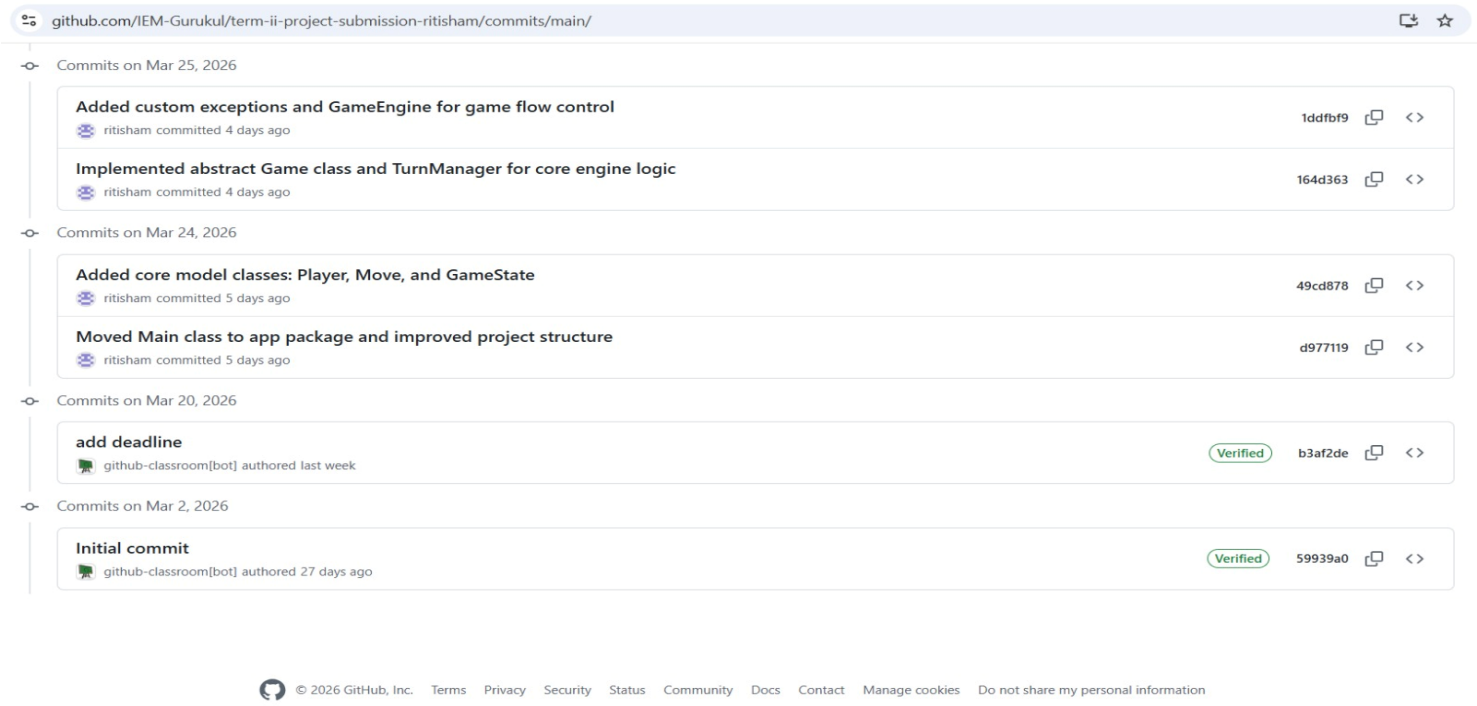
5. TESTING & ERROR HANDLING

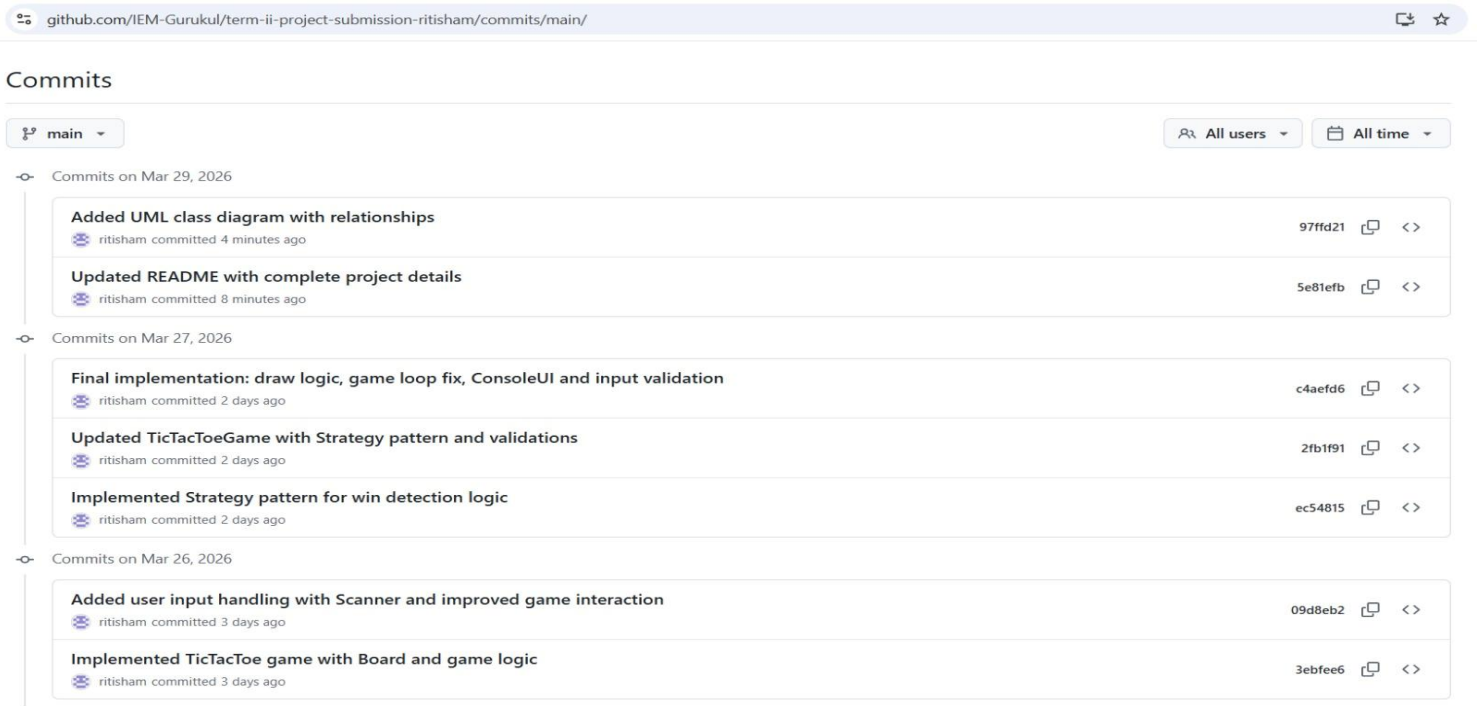
- **Test Cases**
  - *Valid move*
  - *Invalid move*
  - *Same cell*
  - *Win condition*
  - *Draw condition*

- **Edge Cases**
  - *Input out of range*
  - *Full board*
  - *Repeated moves*

- **Failure Handling**
  - *Custom exceptions*
  - *Input validation*
  - *Graceful messages*

6. GIT WORKFLOW





With several significant commits, the project was developed incrementally. Project structure and core class creation were among the first commits. Game logic, design pattern implementation, and UI integration were added in later commits. Bug fixes, input validation, and documentation were the main topics of final commits.

7. CONCLUSION & FUTURE SCOPE

Conclusion:

The project effectively uses OOP concepts to demonstrate a modular and reusable turn-based game engine.

Future Scope:

- *Add GUI*
- *Add multiplayer networking*
- *Add AI opponent*

8. APPENDIX

Appendix A: Initial Project Proposal

Project Title: Multiplayer Turn-Based Game Engine with Java
Problem Statement: Many simple games, such as Tic-Tac-Toe, card games, and strategy board games, have a turn-based structure in which players perform actions sequentially. In most implementations, the game logic and rules are inextricably linked to one game, making it difficult to reuse the code for another game. The goal of this project is to create a tiny Turn-Based Game Engine in Java that handles players, turns, and game data independently of the actual game rules. This engine will allow alternative games to be developed by extending the base classes and creating their own rules. The project focuses on demonstrating good object-oriented architecture, modular structure, and error handling, while maintaining The system is easy to use and expandable.
Target User: Students or novice developers who wish to learn how object-oriented programming principles can be used to organize turn-based multiplayer games.
Core Features:

- Add and oversee several players
- Use the turn management system to decide who gets to play next.
- A depiction of player movements in the game
- Management of game states, including continuing, completed, and winner detection
- An example of how to use the engine to construct a basic game (Tic-Tac-Toe)
- Using exception handling to deal with invalid moves
- Using collections to keep moves and players

OOP Concepts to be Used:

- **Abstraction:** A turn-based game's overall behaviour will be defined by an abstract Game class.

- **Inheritance:** Certain games, such as Tic-Tac-Toe Game, will add their own rules to the underlying Game class.

- **Polymorphism:** Various game implementations will supersede techniques like move validation and winner checking.

- **Exception Handling:** Cases like invalid movements or attempts to play after the game has ended will be handled by custom exceptions

- **Collections:** Players and history will be stored in Java collections such as Array List.

Proposed Architecture Description:
The system will be designed using a modular, object-oriented architecture. By coordinating players, turns, and game state, a central game engine will manage the game's flow. Any turn-based game will have a common structure defined by a base abstract Game class. This class will be expanded with particular games, which will apply their own logic and rules. Player rotation will be handled by a Turn Manager class, while individual classes like Player and Move will represent participants and their actions. This design preserves the engine's reusability and makes it simple to add additional games without altering the fundamental framework.

Appendix B: Faculty Approval Mail

PCCCS495 Proposal Review – APPROVED_MINOR

S

soumadip.biswas@ieminternational.ai
soumadip.biswas@ieminternational.ai

To You Ritisha.Mondal2024@iem.edu.in
Wednesday, 11 March at 8:57 am

Reply-To: soumadip.biswas@ieminternational.ai

Dear Ritisha Mondal,

Project: Multiplayer Turn-Based Game Engine using Java
Decision: APPROVED_MINOR
Score: 92

Strengths:
The proposal clearly identifies a common problem in game development (tightly coupled game logic) and proposes a robust, OOP-driven solu
emphasis on abstraction, inheritance, and polymorphism for creating an extensible game engine is excellent and aligns perfectly with the pro
educational goals. The architectural design, particularly the use of an abstract `Game` class and specific game implementations, demonstr
strong understanding of modular and maintainable software. The features list is comprehensive and appropriate for the scope, and the target
(students/new programmers) is well-defined, making it suitable for a university project. The proposal effectively leverages core Java OOP con

Improvements:
The proposal mentions 'multiplayer' but doesn't specify the scope or implementation details for this feature (e.g., local 'hot-seat' multiplayer
networked multiplayer). Clarifying this upfront would be beneficial, as networked multiplayer would significantly increase the project's comp
While 'Design Pattern' is listed, hinting at specific patterns that might be applied (e.g., Strategy for move validation, Template Method for gam
could further strengthen the architectural description, though this is a minor point for a proposal.

Appendix C: Additional Supporting Screenshots

Game running output

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBr
=== Tic Tac Toe Game Started ===

Player1's turn (X)
Enter row (0-2): 1
Enter col (0-2): 1
- - -
- X -
- - -

Player2's turn (O)
Enter row (0-2): 0
Enter col (0-2): 0
O - -
- X -
- - -

Player1's turn (X)
Enter row (0-2): 2
Enter col (0-2): 0
O - -
- X -
X - -

Player2's turn (O)
Enter row (0-2): 0
Enter col (0-2): 1
O O -
- X -
X - -

Player1's turn (X)
Enter row (0-2): 0
Enter col (0-2): 2
O O X
- X -
X - -
Winner: Player1 (X)
```

```
=== Tic Tac Toe Game Started ===

Player1's turn (X)
Enter row (0-2): 1
Enter col (0-2): 1
- - -
- X -
- - -

Player2's turn (O)
Enter row (0-2): 2
Enter col (0-2): 2
- - -
- X -
- - O

Player1's turn (X)
Enter row (0-2): 2
Enter col (0-2): 4
Error: Invalid position! Enter values between 0 and 2.

Player1's turn (X)
Enter row (0-2):
```

Error handling (invalid input)

```
Enter col (0-2): 1
- O X
- X -
O X O

Player1's turn (X)
Enter row (0-2): 1
Enter col (0-2): 0
- O X
X X -
O X O

Player2's turn (O)
Enter row (0-2): 1
Enter col (0-2): 2
- O X
X X O
O X O

Player1's turn (X)
Enter row (0-2): 1
Enter col (0-2): 1
Error: Cell already occupied!

Player1's turn (X)
Enter row (0-2): 0
Enter col (0-2): 0
X O X
X X O
O X O
Game ended in a draw.
```

Draw condition